



Introduction to Parallelism

Presenter

Anuja Shridhar

Project Engineer

Software Technology group

C-DAC STRATEGIC ELECTRONICS GROUP



C-DAC STRATEGIC ELECTRONICS GROUP



C-DAC STRATEGIC ELECTRONICS GROUP

Parallel Computing

Doing Things Simultaneously

- Same thing or different thing
- Solving one large problem
- Here things refers to programs or tasks (parts of programs).

Task1

Task2

Parallel Computing

Parallel Computing



Why Parallel Computing

Handling complex tasks

E.g. climate modeling, DNA study, machine learning)

Improved performance

E.g. 3D Animations , Pixar studios Toy Story 4

Better resource utilization

E.g. use of GPUs, adobe photo/video editing

Scalability

E.g. Social media, amazon, cloud gaming

Save Time

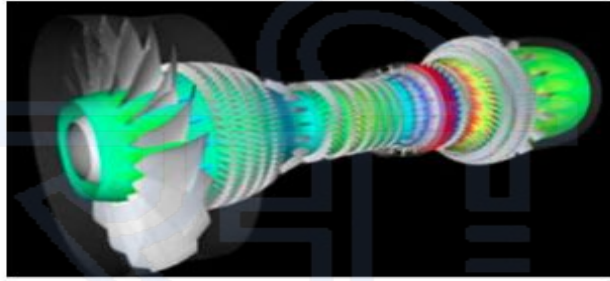
E.g. risk analysis for stock portfolios, search engines

Save Money

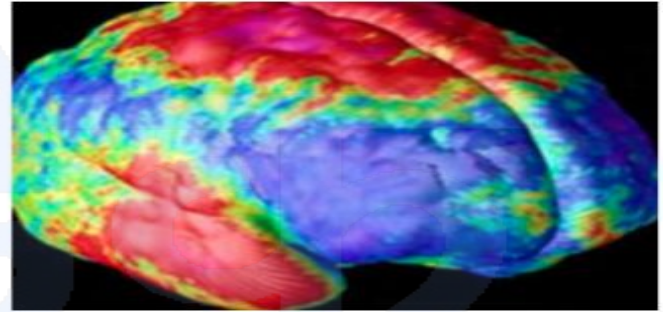
E.g. risk analysis for stock portfolios, search engines

Application of parallel computing

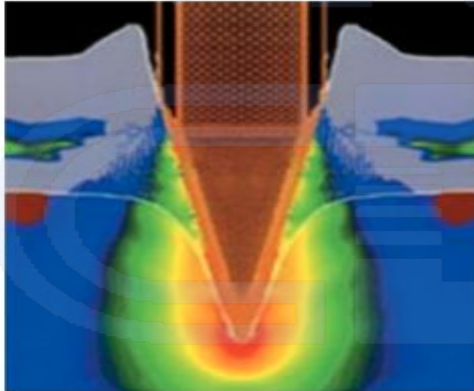
01	Data mining	to extract useful patterns and insights. E.g analyze customer purchase histories and recommend products
02	Real-time simulations	flight simulators, and disaster management. accurate forecasts to help plan for events like hurricanes or floods.
03	Scientific calculations	protein folding, space exploration
04	Graphical application	high-quality graphics in movies, video games, virtual reality.realistic visuals much faster, ensuring shorter production times.
05	AI and Machine learning, etc.	OpenAI's GPT,recommendation engines, and fraud detection



Turbo machinery (Gas turbine/compressor)



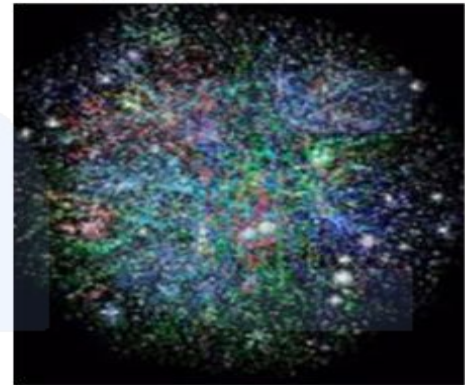
Biology application



Drilling application



Transportation & traffic application



Astrophysics application

Scientific Calculations

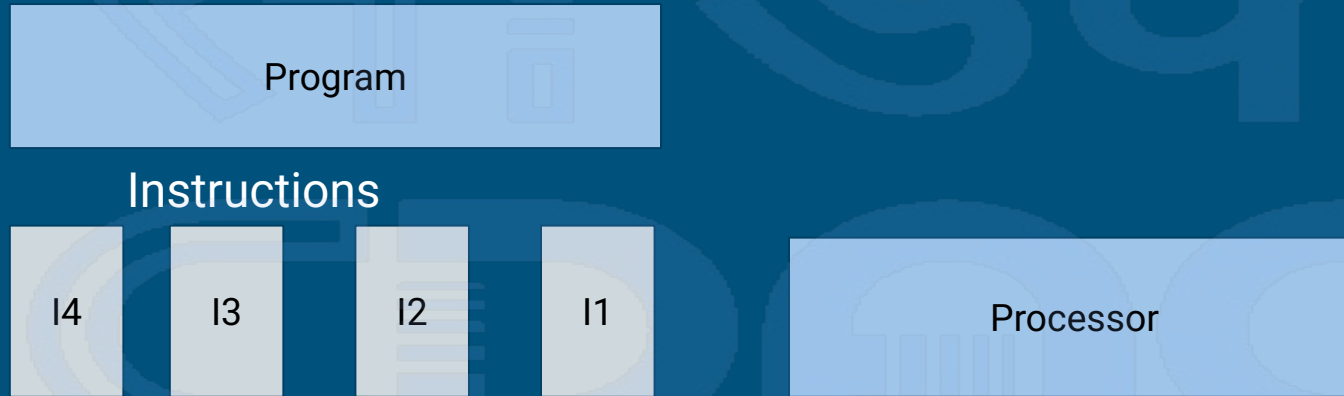
Computing Serial Vs Parallel

C-DAC STRATEGIC ELECTRONICS GROUP

Serial Computing

- Program is broken into stream of instructions that are executed sequentially one after the other on a single processor
- One instruction executes at a time.

Serial Computing



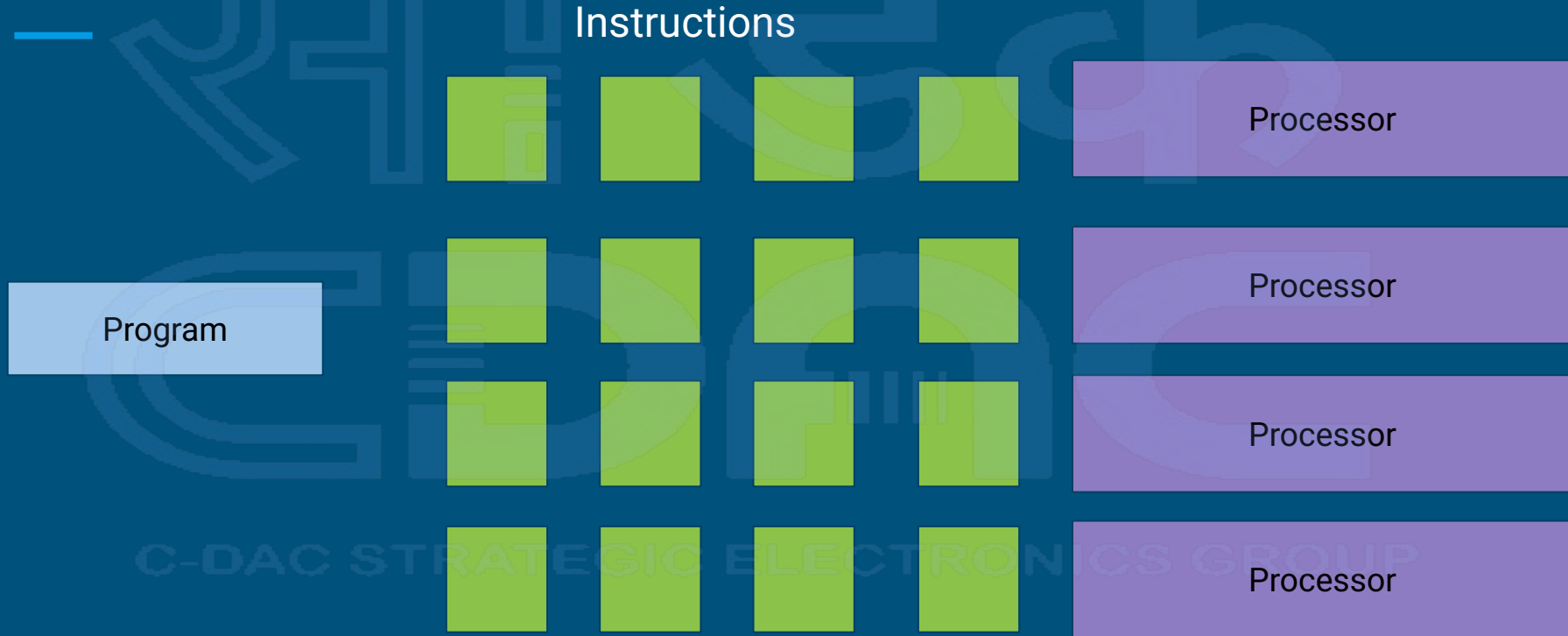
Serial Computing

C-DAC STRATEGIC ELECTRONICS GROUP

Parallel Computing

- Problem is broken into parts that can be solved concurrently
- Each part is further broken into stream of instructions
- Instructions from different parts execute simultaneously on different processors

Parallel Computing



Parallel Computing

Types of Parallel Computing

- Concurrency
- Parallel Execution
- Parallel Concurrent Execution
- Parallelism

Concurrency

Making progress on more than one task -seemingly at the same time.



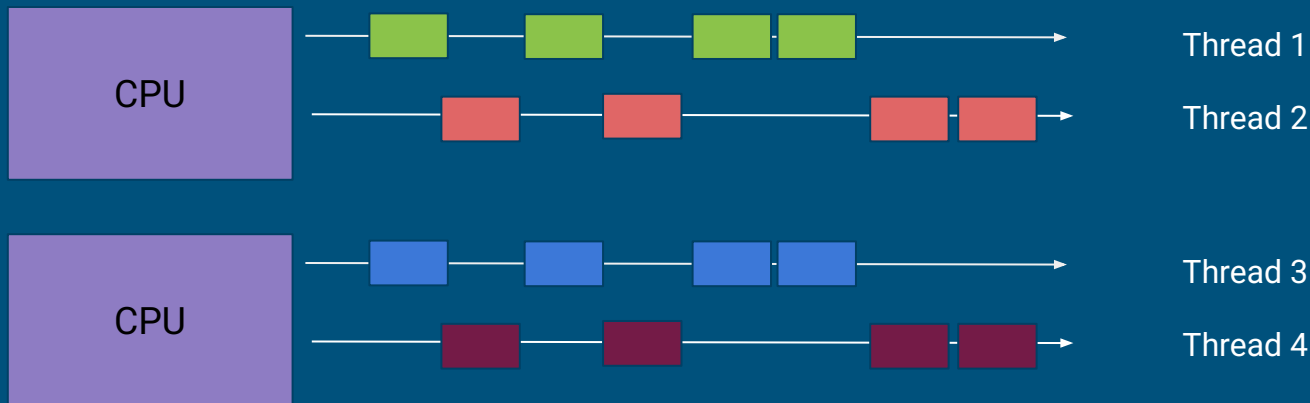
Parallel Execution

Making progression more than one task -seemingly at the same time - on more than one CPU



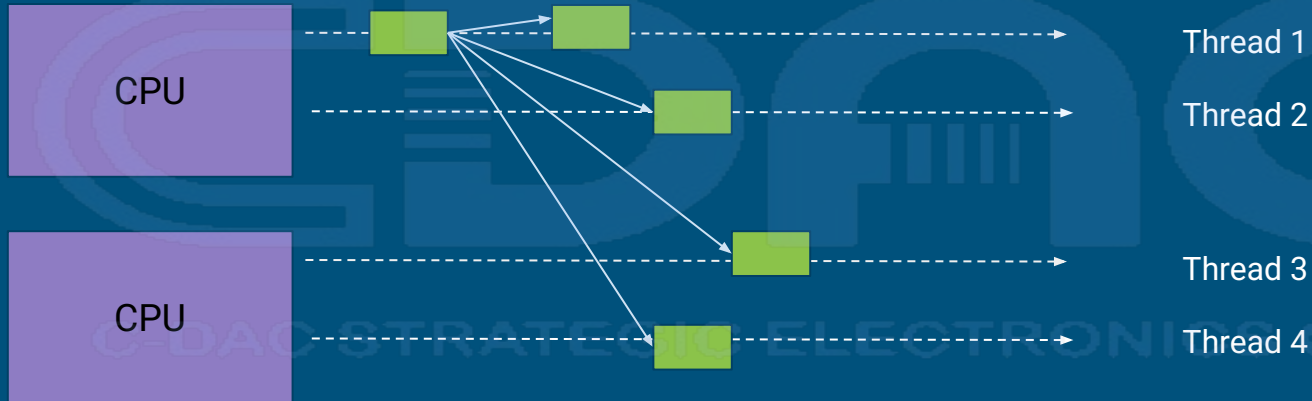
Parallel Concurrent Execution

Making progression more than one task -seemingly at the same time - on more than one CPU



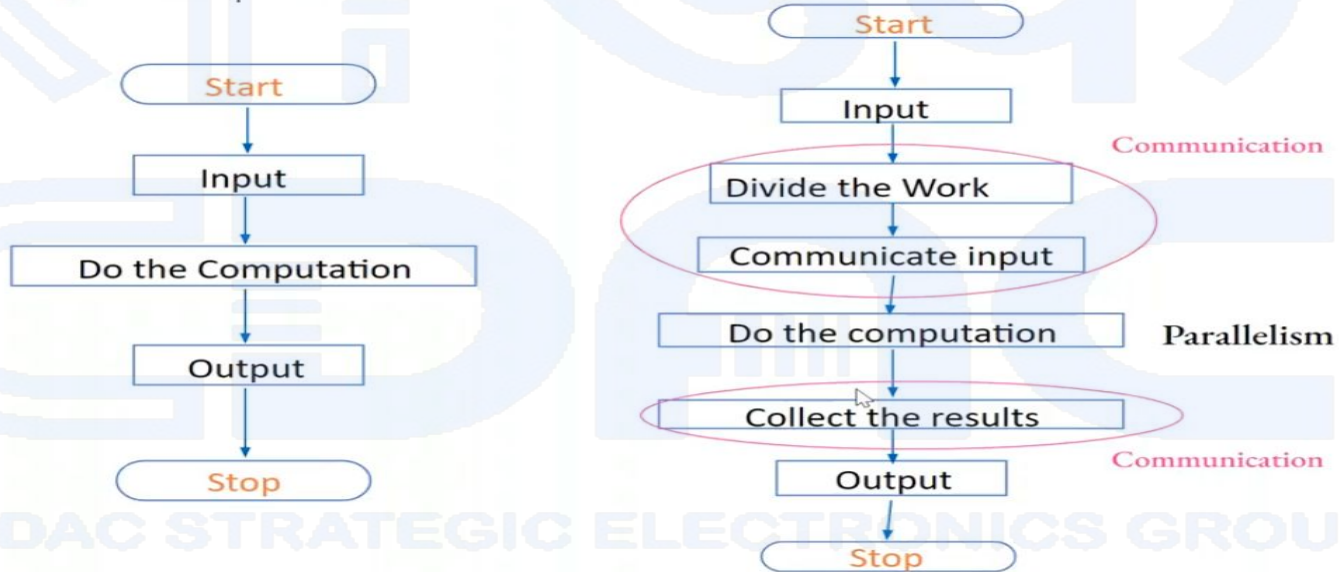
Parallelism

Splitting a single task into subtasks which can be executed in parallel



Program Flow : Serial Vs Parallel

Compute $\sum x_i$ for $(i=1,2,\dots, 10^9)$





How to divide the work in multiple processors

C-DAC STRATEGIC ELECTRONICS GROUP

Types of Parallelism in Computing

- Task Parallelism: Tasks are divided between number of processes/core.
- Data Parallelism: data is divided into no of processes/core. They all perform similar tasks.
- Instruction Level Parallelism : Executing multiple instructions by overlapping their executions.
- Pipeline: Tasks are broken into stages and stages process different inputs concurrently.

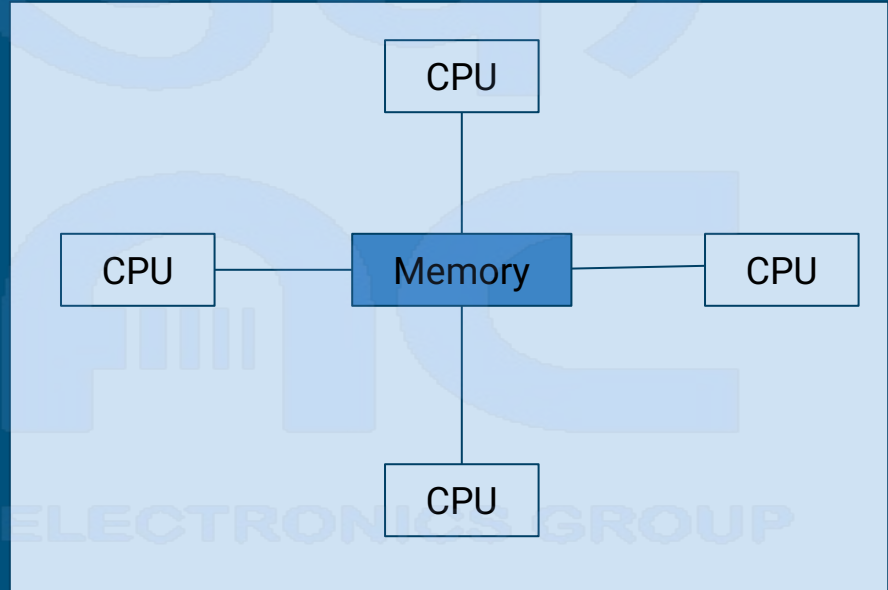
General Parallel computing terminology

- Shared Memory
- Distributed memory
- Node
- Core
- CPU
- Process
- Thread

Shared Memory

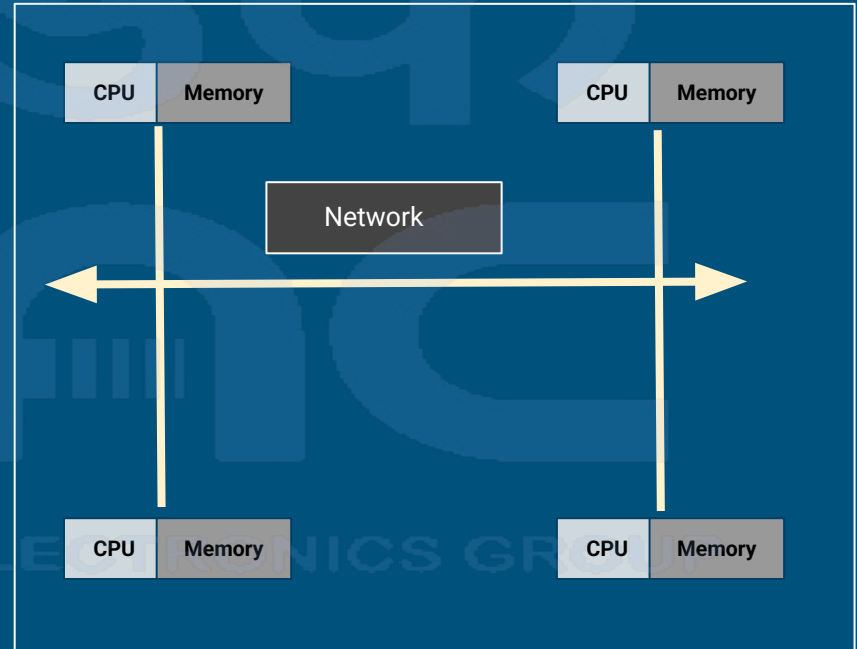
Memory is Shared between multiple
cpu/cores.

OpenMP is used.



Distributed Memory

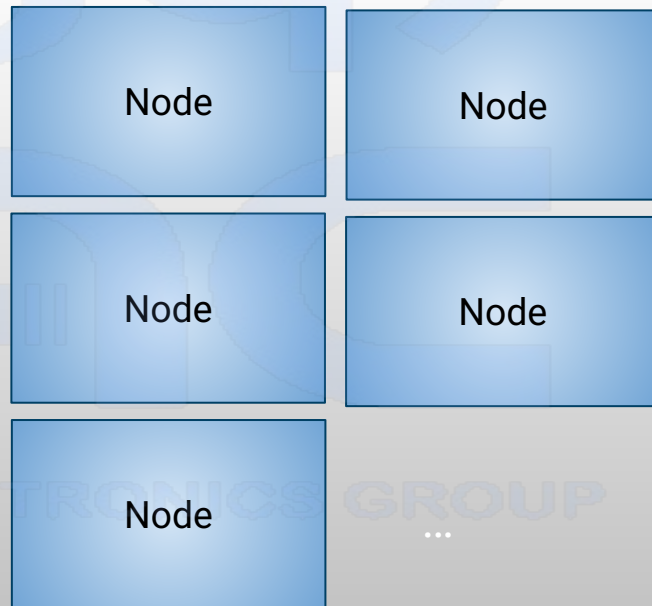
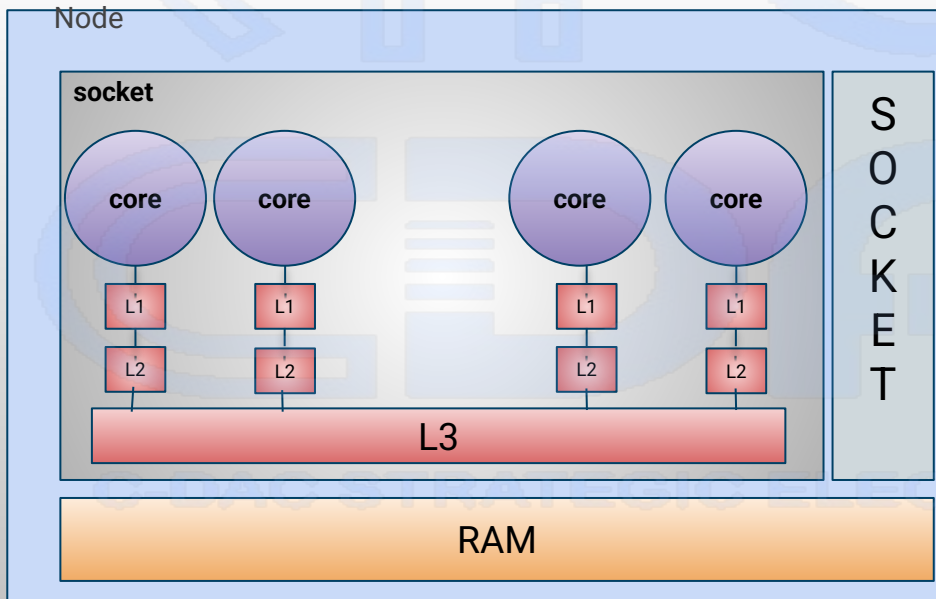
- Every CPU have their own memory space and they are connected through a network
- Message passing is used



Cluster and nodes

Cluster

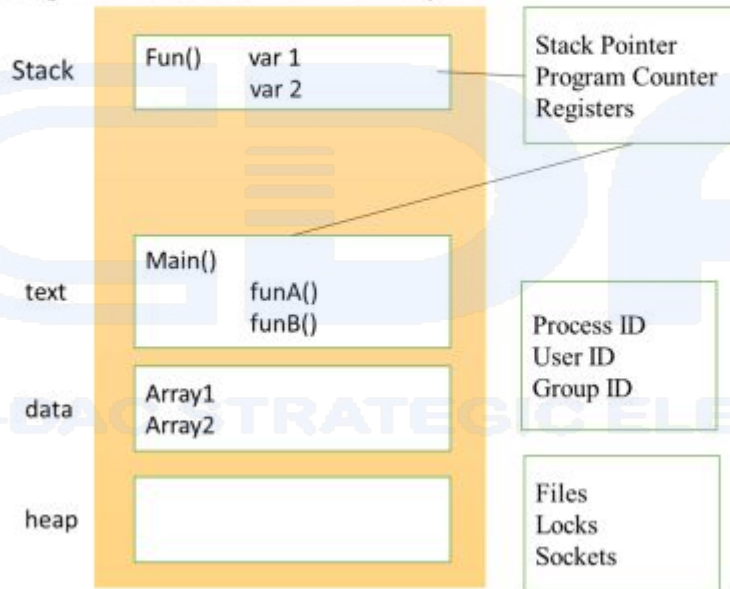
Node



Process and Thread

Process and Threads

- How program execute in memory?

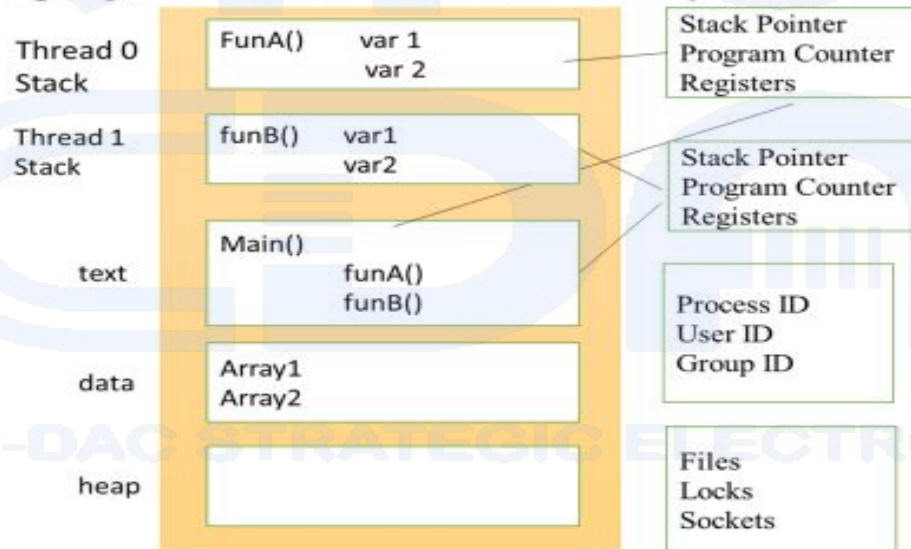


Process:

- An instance of a program execution.
- The execution context of a running program ... i.e. the resources associated with a program's execution.

Process and Threads

- How program execute in shared memory?



Threads:

- Threads are "light weight processes"
- Threads share Process state among multiple threads ... this greatly reduces the cost of switching context.

1. Amdahl's Law

- **Amdahl's Law** : “The speedup of program using multiple processors is limited by the time needed for the sequential fraction of the program.”
 - If an application has fraction **P** that can run in **parallel** and remaining fraction **(1-P)** is **sequential**, the maximum speedup achieved using **N** processors can be formulated as:

$$\text{Speedup}(P, N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

As **N** becomes very large, **P/N** approaches 0 and the speedup tends to $1/(1-P)$.

E.g : Let, 10% of the code is serial in nature and 90% parallelizable & wanted to run this code on 10 processors, So calculate the maximum speedup which can be achieved ?

➤ Here, $N=10$, $P=0.9$, $(1-P)=0.1$ >> $S_P = \frac{1}{0.1 + (0.9) / 10}$

$$= \frac{1}{0.1 + (0.09)} \cong 5$$

As $N \rightarrow \infty$ $S_P \rightarrow 10$

2. Gustafson's Law (aka Gustafson–Barsis's Law)

- In 1988 John L. Gustafson and Edwin H. Barsis wrote an article *Reevaluating Amdahl's Law* showing how application can **retain scalability** on large number of processors by **increasing problem size**
 - If you apply **N** processors to a task that has serial fraction **f**, scaling the task to take the same amount of time as before, the speedup is :

$$\text{Speedup} = f + N(1-f)$$

- Gustafson's law addresses the **shortcomings of Amdahl's law** (which assume fixed problem size) by proposing that programmers can increase size of problems to **fully exploit larger computing systems**.
- In a way the law redefines the **speedup**, due to the possibility that limitations imposed by the sequential part of a program may be countered by increasing the total amount of computation.



Questions

C-DAC STRATEGIC ELECTRONICS GROUP

Questions?

1. What is parallel computing mainly used for?

- A) Reducing power consumption
- B) Increasing computation speed
- C) Making a program more readable
- D) Reducing memory usage

Questions?

Which of the following statements is true?

- A) All parallel programs are concurrent, but not all concurrent programs are parallel
- B) All concurrent programs are parallel, but not all parallel programs are concurrent
- C) Parallel and concurrent programming are the same
- D) Neither parallel nor concurrent programming exists in modern computing

Questions?

Which type of parallelism involves performing the same operation on multiple pieces of data simultaneously?

- A) Task Parallelism
- B) Data Parallelism
- C) Instruction-Level Parallelism
- D) Pipeline Parallelism

Questions?

4. What type of parallelism is mainly utilized in modern CPU architectures for executing multiple instructions per cycle?

- A) Data Parallelism
- B) Task Parallelism
- C) Instruction-Level Parallelism
- D) Bit-Level Parallelism

Questions?

5. Which parallelism type focuses on breaking a problem into multiple tasks that can run independently?

- A) Task Parallelism
- B) Data Parallelism
- C) Bit-Level Parallelism
- D) Serial Processing

Next

How and Who will manage these divided tasks ?

C-DAC STRATEGIC ELECTRONICS GROUP



Need of Parallel Programming

- Handling complex tasks
- Improved performance
- Better resource utilization
- Scalability
- Save time
- Save money



How to do Parallel programming

Problem decomposition

- Task parallelism
- Data parallelism

Task interaction

- Shared memory
- Message passing